

Kuksa System Components and Deployment

This document shows basic KUKSA deployments and gives examples for provider components.

Check the [terminology document](./terminology.md) to get a common understanding for the terms used in this document.

KUKSA aims to provide a consistent view of all signals inside a vehicle. The data model follows the [COVESA Vehicle Signal Specification](https://github.com/COVESA/vehicle_signal_specification) while the data is accessed using either a variant of the [W3C VISS protocol](https://github.com/w3c/automotive) or a [GRPC](https://grpc.io)-based protocol. The KUKSA project develops a Rust VSS Server - the [KUKSA databroker](../).

The following picture shows the basic KUKSA system architecture:

![Basic architecture](./pictures/sysarch_basic.svg)

A KUKSA VSS server runs on a vehicle computer with a given VSS model. Applications can interact with the VSS model by setting, getting or subscribing to VSS datapoints.

The VSS server exposes a network based API to let consumers interact with VSS datapoints. This is either W3C VISS (currently only available in KUKSA server) or the KUKSA GRPC API (currently supported by KUKSA databroker).

Connections to the VSS server can be protected via TLS. Authorization to access specific VSS data is managed via JWT tokens (in KUKSA server, KUKSA databroker implementation is ongoing).

To sync the VSS model to the actual state of the vehicle, providers are used. A data-provider might get its information directly from a sensor, from vehicle busses available to the provider, or from lower parts of the software stack. Historically, data-providers are often called "feeders". To sync the desired state in the VSS server to the Vehicle, actuation-providers are used. Similar to the data-provider, an actuation provider also needs to access lower-level Vehicle systems.

Providers have the same protocol options of accessing the VSS server as consumers.

In the following we will provide some more specific example of consumer and provider components.

Providers Components providing data for leaves in the VSS tree are called **data-providers. Technically they are just normal KUKSA clients. Sometimes you will see data-providers to be referred to as "feeders".**

A data-provider will gather some data from a vehicle using whatever standard or proprietary protocol is needed to access the data. It will then convert the gathered data to the representation defined in a VSS model and provide those standard signals to the VSS server using the VISS or GRPC protocol.

If a provider also supports affecting vehicle systems, based on a desired state being set via the VSS server, it is an **actuation-provider**. Depending on the system design, a provider can be designed to fulfil both the roles as data-provider and actuation-provider.

The following picture shows different kinds of possible KUKSA providers

![[Provider options]](/pictures/sysarch_providers.svg)

We assume running a KUKSA VSS server on a vehicle computer. Some signals might originate in an embedded ECU only connected via CAN (e.g. ECU 1). If the Vehicle Computer running the VSS Server has access to the bus, it can run a provider component to map VSS Datapoints.

The [DBC Feeder](https://github.com/eclipse-kuksa/kuksa-can-provider) is an example of a CAN data-provider. It allows mapping of data from a CAN bus based on a DBC description and some mapping rules.

Other ECUs with Ethernet connectivity might publish data as SOME/IP (ECU 2 in the example) or DDS (ECU 3 in the example) services. The KUKSA project provides an [example SOME/IP provider](https://github.com/eclipse-kuksa/kuksa-someip-provider) based on [vsomeip](https://github.com/COVESA/vsomeip) emulating a SOME/IP controllable wiper.

The Vehicle Computer running the VSS server might run other (automotive) middlewares that provides raw data and signals. In this example we assume the Vehicle Computer runs an AUTOSAR Adaptive subsystem. In that case a provider using the APIs of the underlying system can be created, without the need to parse a specific serialization first. In case of AUTOSAR Adaptive, signals might be accessed using the `ara::com` API.

Finally, there may be other processor based platforms in a vehicle such as another vehicle computer (VCU in the example), a domain, or zone controller or an infotainment system. These systems can run any kind of provider themselves and connect to a VSS server running on a different compute unit directly through the VISS or GRPC directly.

A provider can be implemented directly against the KUKSA GRPC or VISS specification using any programming language. For Python-based providers you can make use of the [KUKSA Python SDK](https://github.com/eclipse-kuksa/kuksa-python-sdk).

Consumers Consumers are usually different kind of applications that are accessing VSS signals through KUKSA. The following figure shows common consumer patterns:

![[Application patterns]](/pictures/sysarch_consumers.svg)

A standard consumer ("Application") will interact with VSS datapoints from the VSS server to realize a vehicle functionality. A simple example is gathering some signals and visualizing them.

Another common use case is providing telemetry to the cloud. With cellular networks, being hidden behind NATs or using dynamic IP addresses, the VISS/GRPC pattern of contacting a VSS server is not a suitable. The common pattern is, that the vehicle pushes relevant data, using suitable IoT protocols and potentially dealing with buffering and connection loss. This is the "Cloud Adapter" pattern.

As a VSS data catalogue can contain signals of different abstraction levels, often higher level signals depend on lower level ones. Thus an application might use data from some signals to calculate the value of others ("Signal Refinement"). An example is, taking current battery voltage and other low level signals from the battery system to calculate a state-of-charge and pushing it back. Another service might use state-of-charge, system and ambient temperature to calculate

remaining range. Such a client combines the roles of consumer and data-provider.

A consumer can be implemented directly against the KUKSA GRPC or VISS specification using any programming language. For Python-based consumers you can make use of the [KUKSA Python SDK](https://github.com/eclipse-kuksa/kuksa-python-sdk).

(Distributed) KUKSA deployment

The idea behind VSS and KUKSA is, to provide a single entry point to all vehicle data. As in a real vehicle data is distributed and not all domains need access to all data. Therefore, you might opt to run several VSS servers, each providing a subset of all vehicle data. This can also increase resiliency of the system and allows to separate safety domains.

What is considered a suitable deployment is very much dependent on the vehicle architecture and the scope of data managed in VSS.

The following figure shows several possible patterns:

![[Deployment patterns](./pictures/sysarch_deployment.svg)]

The simplest expected deployment assumes that the primary vehicle computer contains the fully populated VSS model for a vehicle. This is the only possible deployment in vehicles, that only have one computer capable to run KUKSA and protocols such as GRPC or VISS.

In a redundancy scenario there may be a backup vehicle computer with a KUKSA instance that is kept in sync, so that in case of a failover, the last known data is immediately available for all VSS datapoints. We assume an optimized VSS Sync component to be used that might deal with one-way or two-way syncs. Currently KUKSA does not include a sync mechanism.

A vehicle can have additional domain controllers such as the powertrain controller. The Powertrain controller might use VSS to manage private internal datapoints not relevant to other domains. At the same time it is accessing VSS signals relevant to powertrain functionality provided by the VSS server on the central vehicle computer as well as providing higher level powertrain signals to the vehicle computer.

The Infotainment/Displays example extends this pattern, where the system subscribes needed data from the central Vehicle Computer (e.g. data that needs to be visualized), but also wants to actuate things in the vehicle via VSS model (e.g. if a driver sets charge limits in the UI).